

Programación

III – TP Final

Food Store

- Alumnos:
 - Nahuel Urciuoli Zabala – Nahuel_zabala@live.com
- Título del Proyecto:
 - Food Store - Sistema Integral de Gestión y Venta de Alimentos
- Fecha de entrega:
 - 24/06/2025

Índice

Marco Teórico.....	3
Decisiones Técnicas y Arquitectura del Sistema.....	3
Dificultades Encontradas y Soluciones Aplicadas	14
Bibliografía	15

Marco Teórico

Java 23 & Spring Boot: Se utilizó Java como lenguaje principal en el backend debido a su fuerte tipado y madurez en entornos corporativos. Spring Boot simplificó la configuración inicial mediante el uso de starter-packs, permitiendo levantar una API REST funcional de manera ágil.

JPA / Hibernate: Como solución de persistencia (ORM), se implementó Jakarta Persistence API (JPA) a través de Hibernate. Esto permitió abstraer las consultas SQL nativas, mapeando las tablas de la base de datos directamente a clases de Java (Entidades), optimizando la mantenibilidad del código.

H2 Database: Durante la etapa de desarrollo y testing se optó por una base de datos H2 en memoria. Esto facilitó pruebas rápidas sin necesidad de configuraciones complejas de servidores locales, asegurando un entorno limpio en cada ejecución.

Vite & Frontend Stack: En el desarrollo de la interfaz de usuario se utilizó Vite como empaquetador debido a su velocidad de compilación y recarga en caliente (HMR), consumiendo la API de Spring mediante peticiones asíncronas (Fetch API).

Decisiones Técnicas y Arquitectura del Sistema

El backend se diseñó siguiendo el patrón de diseño de Arquitectura en Capas para asegurar la separación de responsabilidades:

Capa de Servicio (Service): Contiene la lógica de negocio central del sistema (validación de stock, etc.).

Capa de Acceso a Datos (Repository): Interfaces que extienden de JpaRepository para la comunicación directa con la base de datos.

Patrones Aplicados

Uso de DTOs (Data Transfer Objects): Se decidió implementar DTOs para desacoplar las entidades de la base de datos de las respuestas de las Queryx. Esto evita exponer datos sensibles.

Ejemplo de DTO

Se guarda el usuario en la BD haciendo uso de Usuario repository

```
public interface UsuarioRepository extends JpaRepository<Usuario, Long> {  
  
    Optional<Usuario> findByMail(String email); 1 usage  
    boolean existsByMail(String email); 2 usages
```

```
public interface UsuarioService { 4 usages 1 implementation
    public UsuarioDto save (UsuarioCreate usuarioCreate); 1 implementation
    public UsuarioDto findById(Long Id); 1 implementation
    public List<UsuarioDto> findAll(); 1 implementation
    public UsuarioDto update(UsuarioEdit usuarioEdit, Long idUsuario); 1 implementation
    public void deleteById(Long id); 1 implementation
    public UsuarioDto findByMail(String mail); 1 usage 1 implementation
}
```

Se extiende del Service y se agrega el repositorio del Usuario para poder hacer uso de los métodos de JPA repository para el acceso a la BD y se devuelve la respuesta en un DTO de usuario

```
@RequiredArgsConstructor
@Service
public class UsuarioServiceImpl implements UsuarioService{
    private final UsuarioRepository usuarioRepository;

    @Override
    public UsuarioDto save(UsuarioCreate usuarioCreate) {
        if (usuarioRepository.existsByMail(usuarioCreate.mail())) {
            throw new IllegalArgumentException("Ya existe un email con dirección " + usuarioCreate.mail());
        }
        Usuario usuario = usuarioCreate.toEntity();
        usuario = usuarioRepository.save(usuario);

        return UsuarioDto.toDto(usuario);
    }
}
```

No exponiendo datos sensibles como por ejemplo la contraseña del usuario en la respuesta

```
public record UsuarioDto( 34 usages
    Long id, 2 usages
    String nombre, 4 usages
    String apellido, 4 usages
    String mail, 4 usages
    String celular, 3 usages
    Boolean activo, no usages
    Rol rol, 1 usage
    List<PedidoDto> pedidos 3 usages
) {
    public static UsuarioDto toDto(Usuario usuario) { 5 usages
        return new UsuarioDto(
            usuario.getId(),
            usuario.getNombre(),
            usuario.getApellido(),
            usuario.getMail(),
            usuario.getCelular(),
            usuario.getActivo(),
            usuario.getRol(),
            //Se fija si pedidos del usuario son null si son null crea lista vacia
            usuario.getPedidos() != null
                ? usuario.getPedidos().stream().map(PedidoDto::toDto).toList()
                : List.of()
        );
    }
}
```

Para el front end se uso un carrito que persiste en LocalStorage comparando siempre el elemento en el local Storage con los datos del JSON en todo momento.

El usuario persiste en el Session Storage

Storage	Key	Value
Local storage	usuarioLogueado	{"id":2,"nombre":"Juan","apellido":"Pérez"}
Session storage		
http://localhost:5173		

Mientras que el carrito persiste en LocalStorage

Storage	Key	Value
Local storage	cart	[{"producto":{"id":1,"nombre":"Hamburguesa Clásica","precio":4500,"o"}
Session storage		
http://localhost:5173		

Antes de iniciar cada pagina se verifica que la persona tenga una sesión iniciada

Si rol requerido no es el adecuado se manda a home en caso de ser Usuario se manda al home de usuario

```
export const verificarUsuario : (rolRequerido: Rol) => void = (rolRequerido: Rol) : void =>
  const user : string | null = getUser()
  if (!user) {
    console.log( message: "no existe en local");
    navigate( route: "/src/pages/auth/login/login.html");
    return;
  } else {
    const parseUser: IUser = JSON.parse(user);
    if(parseUser.rol=="ADMIN") {
      return;
    }
    if (parseUser.rol !== rolRequerido) {
      if (parseUser.rol === "USUARIO" ) {
        navigate( route: "/src/pages/store/home/home.html")
        return
      }
      else {
        navigate( route: "/src/pages/admin/home/home.html");
        return
      }
    }
  }
}
```

Ejemplo de Panel Admin

```
> import ...

verificarUsuario( rolRequerido: "ADMIN")

const productoTodos: Product[] = productos
const categoriasTodas: ICategory[] = categorias
const pedidosTodos: IPedido[] = pedidos

const barraLateral = document.querySelector( selectors: ".sidebar-menu") as HTMLDivElement;
const contenedorGeneral = document.querySelector( selectors: ".main-content") as HTMLElement;
```

Inicio de sesión

SuperFood

Iniciar Sesión

Email

password

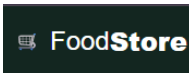
Ingresar

Cuentas de prueba:

Admin: admin@admin.com / 123456

Cliente: cliente@food.com / cliente123

Pagina home



CatalogoMis PedidosCarrito1Juan PérezCerrar Sesión

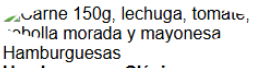
CATEGORÍAS

Todos los productos

HamburguesasPizzasEmpanadasBebidasPapas Fritas

Buscar productos...

Ordenar por: Destacados



Carne 150g, lechuga, tomate, cebolla morada y mayonesa

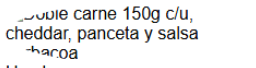
Hamburguesas

Hamburguesa Clásica

Carne 150g, lechuga, tomate, cebolla morada y mayonesa

\$4500

+ Agregar



Doble carne 150g c/u, cheddar, panceta y salsa barbacoa

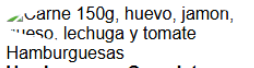
Hamburguesas

Hamburguesa Doble

Doble carne 150g c/u, cheddar, panceta y salsa barbacoa

\$6200

+ Agregar



Carne 150g, huevo, jamon, queso, lechuga y tomate

Hamburguesas

Hamburguesa Completa

Carne 150g, huevo, jamón, queso, lechuga y tomate

\$5500

+ Agregar

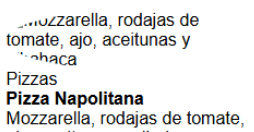


Mozzarella, salsa de tomate y orégano

Pizzas

Pizza Mozzarella

Mozzarella, salsa de tomate y orégano



Mozzarella, rodajas de tomate, ajo, aceitunas y albahaca

Pizzas

Pizza Napolitana

Mozzarella, rodajas de tomate, ajo, aceitunas y albahaca

Filtrado de producto por categoría



CatalogoMis PedidosCarrito1Juan PérezCerrar Sesión

CATEGORÍAS

Todos los productos

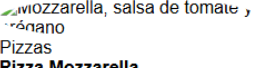
Hamburguesas

Pizzas

EmpanadasBebidasPapas Fritas

Buscar productos...

Ordenar por: Destacados



Mozzarella, salsa de tomate y orégano

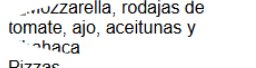
Pizzas

Pizza Mozzarella

Mozzarella, salsa de tomate y orégano

\$3800

+ Agregar



Mozzarella, rodajas de tomate, ajo, aceitunas y albahaca

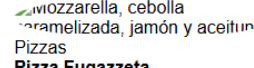
Pizzas

Pizza Napolitana

Mozzarella, rodajas de tomate, ajo, aceitunas y albahaca

\$4500

+ Agregar



Mozzarella, cebolla caramelizada, jamón y aceitunas

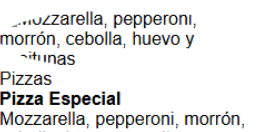
Pizzas

Pizza Fugazzeta

Mozzarella, cebolla caramelizada, jamón y aceitunas

\$4800

+ Agregar



Mozzarella, pepperoni, morrón, cebolla, huevo y aceitunas

Pizzas

Pizza Especial

Mozzarella, pepperoni, morrón, cebolla, huevo y aceitunas

Filtrado por búsqueda (Descripción y nombre producto)

 hamb

Carne 150g, lechuga, tomate,
cebolla morada y mayonesa

Hamburguesas
Hamburguesa Clásica

Carne 150g, lechuga, tomate,
cebolla morada y mayonesa

\$4500

+ Agregar

Doble carne 150g c/u,
cheddar, panceta y salsa
barbacoa

Hamburguesas

Hamburguesa Doble

Doble carne 150g c/u, cheddar,
panceta y salsa barbacoa

\$6200

+ Agregar

Ordenamiento A-Z

 hamb

Ordenar por: Nombre (A-Z)



Bebidas
Agua Mineral 500ml
Agua mineral sin gas 500ml

\$1000

+ Agregar



Papas Fritas
Aros de Cebolla
Aros de cebolla empanizados,
servidos con salsa barbacoa

\$2800

+ Agregar



Bebidas
Cerveza Artesanal 473ml
Cerveza artesanal rubia 473ml

\$2500

+ Agregar

Precio

hamb

Ordenar por:

Precio: Menor a Mayor



Bebidas
Agua Mineral 500ml
Agua mineral sin gas 500ml

\$1000

+ Agregar



Empanadas
Empanada Jamón y Queso
Jamón cocido y mozzarella

\$1100

+ Agregar



Empanadas
Empanada de Verdura
Espinaca, acelga, ricota y nuez moscada

\$1100

+ Agregar

Producto sin stock no permite agregar al carrito

Carne 150g, lechuga, tomate, cebolla morada y mayonesa
Hamburguesas
Hamburguesa Clásica
Carne 150g, lechuga, tomate, cebolla morada y mayonesa

\$4500

Sin stock

Carrito con total y agregar más productos

Mi Carrito

Carne 150g, lechuga, tomate, cebolla morada y mayonesa

Hamburguesa Clásica
Precio: \$4500

- 3 +
Eliminar

Doble carne 150g c/u, cheddar, panceta y salsa barbecue

Hamburguesa Doble
Precio: \$6200

- 1 +
Eliminar

Carne 150g, huevo, jamón, queso, lechuga y tomate

Hamburguesa Completa
Precio: \$5500

- 1 +
Eliminar

Resumen

Subtotal: \$25200

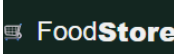
Total: \$25200

Finalizar Compra

Boton de checkout desactivado en esta version

Vaciar carrito

Menús pedidos con filtro según estado

CatalogoMis PedidosCarrito0Juan PérezCerrar Sesión

Mis Pedidos

Todos

Pedido #1

2026-06-08

Hamburguesa Clásica (x2)

Agua Mineral 500ml (x2)

2 producto(s)

\$9600.00

Pedido #2

2026-06-09

Hamburguesa Doble (x1)
Pizza Especial (x1)

Papas Fritas Clásicas (x2)

3 producto(s)

\$16400.00

Pedido #3

2026-06-10

Empanada de Carne (x3)

Cerveza Artesanal 473ml (x1)

2 producto(s)

\$6100.00

Al hacer clic se abre modal para desplegar detalles

Mis Pedidos

Todos

Pedido #1

2026-06-08

Hamburguesa Clásica (x2)

2 producto(s)

\$9600.00

Pedido #2

2026-06-09

Hamburguesa Doble (x1)
Pizza Especial (x1)

3 producto(s)

\$16400.00

Pedido #3

2026-06-10

Empanada de Carne (x3)

2 producto(s)

\$6100.00

ENTREGADO

2026-06-08

Información de Entrega

Teléfono: 1198765432

Método de pago: TARJETA

Productos

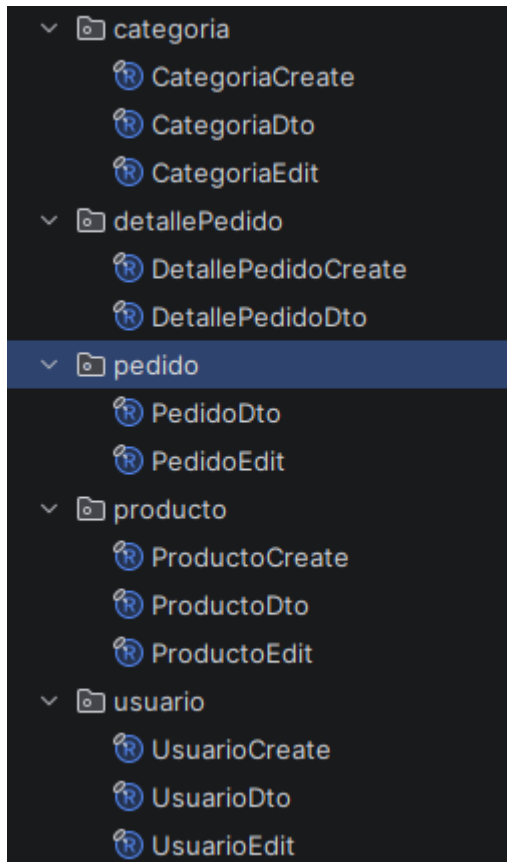
Hamburguesa Clásica	\$9000.00
Cantidad: 2 × \$4500.00	
Agua Mineral 500ml	\$600.00
Cantidad: 2 × \$1000.00	
Total:	\$9600.00

Tu pedido está siendo procesado

Te notificaremos cuando sea confirmado.

Back-end:

DTO de las entidades:



Ejemplo de DTO Producto

con método toEntity para pasar productos creados (pedidoCreate) a una entidad de Producto

```
@Entity
@Getter
@Setter

@Builder
@ToString
public class Producto {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nombre;
    private BigDecimal precio;
    private String descripcion;
    private int stock;
    private String imagen;
    private Boolean disponible;

    @ManyToOne
    @JoinColumn(name="categoria_id")
    private Categoria categoria;

    public Producto(String nombre, BigDecimal precio, String descripcion, int stock, String imagen, Categoria categoria) {
        this(nombre, precio, descripcion, stock, imagen, categoria, disponible = true);
    }

    public Producto(String nombre, BigDecimal precio, String descripcion, int stock, String imagen, Categoria categoria, boolean disponible) {
        this.nombre = nombre;
        this.precio = precio;
        this.descripcion = descripcion;
        this.stock = stock;
        this.imagen = imagen;
        this.categoria = categoria;
        this.disponible = disponible;
    }
}
```

```
@Builder 11 usages
public record ProductoCreate(

    @NotBlank(message = "El nombre no puede estar vacio") 1 usage
    String nombre,

    @NotNull(message = "El precio no puede ser nulo") 1 usage
    @PositiveOrZero(message = "El precio debe ser mayor o igual a 0")
    BigDecimal precio,

    String descripcion, 1 usage

    @PositiveOrZero(message = "El stock no puede ser negativo") 1 usage
    int stock,

    String Imagen, 1 usage
    Boolean disponible, no usages

    @NotNull(message = "La categoria es obligatoria") 1 usage
    Long idCategoria
) {
    public Producto toEntity(Categoria categoria){ 1 usage
        return new Producto(this.nombre,this.precio,this.descripcion,this.stock,this.Imagen,categoria);
    }
}
```

El usuario ingresa los datos del Producto que quiera crear se validan (Ej. stock>0) si son validos se crea el Producto y se llama al productService con la función “save” para validar datos y persistirlos en la BD

```
private static void altaProducto() { 1 usage
    System.out.println("\n Alta de Producto");
    listadoCategorias();
    System.out.print("Ingrese ID de Categoria: ");
    try {
        Long catId = Long.parseLong(scanner.nextLine());
        categoriaService.findById(catId);
        boolean disponible = true;
        System.out.print("Nombre Producto: ");
        String nombre = scanner.nextLine().trim();
        System.out.print("Descripción: ");
        String desc = scanner.nextLine().trim();
        System.out.print("Precio (> 0): ");
        BigDecimal precio = new BigDecimal(scanner.nextLine().trim());
        System.out.print("Stock (>= 0): ");
        int stock = Integer.parseInt(scanner.nextLine());
        System.out.print("Disponible? S/N: ");
        String disponibleString = scanner.nextLine().trim();
        if (disponibleString.equalsIgnoreCase("n")) {
            disponible = false;
        } else if (!disponibleString.equalsIgnoreCase("s")) {
            System.out.println("Ingrese datos validos S o N " + disponibleString + " no es un dato valido");
            return;
        }
        if (precio.compareTo(BigDecimal.ZERO) <= 0 || stock < 0 || nombre.isEmpty()) {
            System.out.println("ERROR: Restricciones de validación no superadas.");
            return;
        }
        ProductoCreate prod = ProductoCreate.builder().nombre(nombre).disponible(disponible).idCategoria(catId).descripcion(desc).Imagen("Imagen.jpg").precio(precio).stock(stock).build();

        ProductoDto productoBD = productoService.save(prod);
        System.out.println("Producto creado con ID: " + productoBD.id() + " en Categoria: " + productoBD.categoriaDto().nombre());
    } catch (NumberFormatException e) {
        System.out.println("Error ingreso datos validos");
    } catch (NullPointerException ex) {
        System.out.println(ex.getMessage());
    }
}
```

El service usa los métodos de el Repositorio de Categoría y Producto para poder usar los métodos que vienen en el JPA repository y trabajar directamente con la BD (save/findById/Update etc)

```
public class ProductoServiceImpl implements ProductoService {
    private final ProductoRepository productoRepository;
    private final CategoriaRepository categoriaRepository;
    private final CategoriaService categoriaService;

    @Override
    public ProductoDto save(ProductoCreate productoCreate) {
        //No se agrega el producto si el id de la categoria no existe
        Categoria categoria = categoriaRepository.findById(productoCreate.idCategoria()).orElseThrow(() -> new NullPointerException("No se encontro categoria"));
        if(categoria.getEliminado()){
            throw new IllegalArgumentException("Categoria no existe o eliminada de BD");
        }
        //creo el producto y le agrego la categoria al final
        Producto producto = productoCreate.toEntity(categoria);
        //Te lo devuelve con el ID generado por la BD
        producto = productoRepository.save(producto);
        return ProductoDto.toDto(producto);
    }
}
```

Repository:

```
@Repository 5 usages
public interface CategoriaRepository extends JpaRepository<Categoria, Long> {

    boolean existsByNombre(String nombre); 1 usage

}
|
```

Si el proceso se realiza con éxito y no se ejecuta ninguna excepción se imprime mensaje de éxito mostrando el Producto con el ID generado por la BD

```
System.out.println("Producto creado con ID: " + productoBD.id() + " en Categoria: " + productoBD.categoriaDto().nombre());
} catch (NumberFormatException e) {
    System.out.println("Error ingreso datos validos");
} catch (NullPointerException ex) {
    System.out.println(ex.getMessage());
}
```

Dificultades Encontradas y Soluciones Aplicadas

Dificultad: Persistencia distribuida y sincronización de datos.

Solución: Se logró estabilizar el mapeo de relaciones Complejas en JPA utilizando anotaciones correctas de FetchType y cascadas para evitar la carga perezosa (LazyInitializationException) al mapear pedidos

Dificultad: Al delegar la lógica de negocio a los servicios (productoService, categoriaService), las búsquedas fallidas (como buscar un ID inexistente) lanzaban excepciones directas al menú de la consola (ej. NullPointerException o NoSuchElementException). Al no estar controladas en la interfaz de usuario, estas excepciones provocaban que la aplicación colapsara por completo y se apagara en medio de una operación.

Solución: Se implementó un esquema de arquitectura defensiva en la clase principal mediante bloques try-catch específicos para cada operación del menú. Se identificó que soluciones globales como @ControllerAdvice son exclusivas para entornos Web (HTTP API), por lo que se adaptó el

control capturando manualmente excepciones numéricas (`NumberFormatException`), argumentos inválidos (`IllegalArgumentException`) y errores generales, logrando que el sistema informe el error amigablemente en pantalla y retorne al usuario de forma segura al menú principal.

Dificultad: Al listar o actualizar productos que carecían de una categoría asignada (o que apuntaban a registros huérfanos), el mapeo hacia los DTOs fallaba. Intentar invocar métodos encadenados como `p.categoriaDto().id()` o comprobar la disponibilidad de la categoría de forma directa provocaba errores críticos de tipo `NullPointerException`.

Solución: Se implementaron validaciones de cortocircuito lógico utilizando operadores condicionales ternarios y orden de precedencia estricto (evaluando siempre `!= null` en el extremo izquierdo). De esta forma, si el producto no cuenta con categoría, el sistema mitiga el error en tiempo de ejecución asignando valores de contingencia seguros por defecto (ej. ID en 0L y nombres como "Sin categoría"), garantizando la estabilidad del listado

Dificultad: Al implementar la modificación de un producto, se buscaba que el usuario pudiera presionar Enter para conservar el valor actual de un campo. Sin embargo, al parsear directamente la consola hacia tipos nativos u objetos numéricos complejos (`Integer.parseInt` o `new BigDecimal`), las entradas vacías ("") hacían estallar el programa. Además, declarar las variables validadas dentro de bloques internos `if` restringía su alcance (scope), impidiendo su uso al construir el DTO de edición al final del método.

Solución: Se reestructuró la lectura de consola capturando inicialmente todas las entradas como cadenas de texto (`String`). Se inicializaron las variables de actualización con los datos vigentes del DTO recuperado de la base de datos. De este modo, si la entrada del scanner detecta un vacío (`.isEmpty()`), el sistema ignora de forma segura el bloque de conversión, manteniendo intacto el valor original del registro y permitiendo una edición parcial y limpia de los campos.

Bibliografía

Hibernate. *ORM User Guide*. <https://hibernate.org/orm/documentation/>

Google. (2026). *Gemini*. <https://gemini.google.com/>

Conversaciones: <https://gemini.google.com/share/86f7cbcc7d71>

Spring. *Spring Framework Documentation*. <https://spring.io/docs>